SLE-4: Final Viva & Architecture Decision Record (ADR)

Course: 02AML204 – Introduction to Artificial Intelligence

PRN: 25UAM037

Name: Pallavi Vasant Koli

Division: A

Project Title: BFS vs DFS Search System

GitHub Link: https://github.com/pallavikolii285-ops/IAI_SLE2.git

https://github.com/pallavikolii285-ops/IAI_SLE3.git

Part A — Architecture Decision Record (ADR)

ADR Title: Use of a Modular Design for Comparing BFS and DFS.

Status: Accepted

Date: 11 October 2026

1. Context

My project is a Python-based BFS vs DFS Search System. It searches a 20-node graph from start node 0 to goal node 19 using Breadth-First Search (BFS) and Depth-First Search (DFS). BFS uses a queue, while DFS uses a stack. In SLE-2, I measured execution time and expanded nodes using timeit and generated a py-spy profile. In SLE-3, I documented the system architecture using the four C4 levels.

2. Decision

I decided to keep BFS and DFS in a modular search system, using the same graph and start/goal nodes for a fair comparison. The system separates graph/configuration, search, performance measurement and result/output responsibilities.

3. Alternatives Considered

- **Alternative 1:** Use only BFS. This allows level-by-level search but does not allow comparison with DFS.
- **Alternative 2:** Use only DFS. This removes the opportunity to compare it with BFS.
- **Alternative 3:** Keep both algorithms in a modular system. This supports fair comparison and improves organization and maintenance.

SLE-2: Empirical Performance Analysis

In SLE-2, I performed an empirical performance analysis of Breadth-First Search (BFS) and Depth-First Search (DFS). The objective was to compare both algorithms based on execution time and the number of nodes expanded. I used a graph containing 20 nodes, with node 0 as the start node and node 19 as the goal node. Both algorithms were tested using the same graph and search configuration. I used Python's `timeit` module to measure execution time and also used `py-spy` for profiling. The experiment was conducted over three runs, and the results were compared.

Performance Comparison of BFS and DFS (SLE-2)

Metric	BFS	DFS
Graph size	20 nodes	20 nodes
Start node	0	0
Goal node	19	19
Average execution time	9.45187 ms	4.35503 ms
Average nodes expanded	20	6
Goal found	True	True
Better performance in this experiment	—	DFS

The results show that DFS recorded a lower average execution time than BFS. DFS expanded 6 nodes, while BFS expanded 20 nodes. Both algorithms successfully found the goal node. Therefore, DFS performed better in this particular experiment. However, the results depend on the graph structure and goal position, so DFS may not always perform better than BFS.

SLE-3: Architectural Design Using the Full C4 Model

In SLE-3, I designed the architecture of the BFS vs DFS Search System using the full C4 model. The C4 model represents the system at four levels: Context, Container, Component, and Code. The Context diagram shows how the user interacts with the system. The Container diagram describes the main modules, while the Component diagram explains the internal parts of the search engine. The Code level identifies the main functions and their responsibilities.

C4 Architecture Levels Used in SLE-3

C4 Level	What It Explains in My Project

Level 1 – Context	Shows the user interacting with the BFS vs DFS Search System and receiving the search results.
Level 2 – Container	Shows the main modules: Graph/Configuration Module, Search Engine, Performance Measurement, and Result/Output Module.
Level 3 – Component	Shows the internal search components, including the BFS queue, DFS stack, visited set, goal test, and node counter.
Level 4 – Code	Identifies the main functions and operations, such as `bfs()`, `dfs()`, execution-time measurement, average calculation, and result display.

The architecture separates graph configuration, search operations, performance measurement, and output responsibilities. BFS uses a queue to manage nodes waiting for exploration, whereas DFS uses a stack. The visited set helps avoid repeated exploration, and the goal test checks whether the required node has been reached. The performance measurement module records execution time, while the output module displays the results. This modular architecture makes the system easier to understand, test, and maintain. It also supports the architectural decision documented in SLE-4.

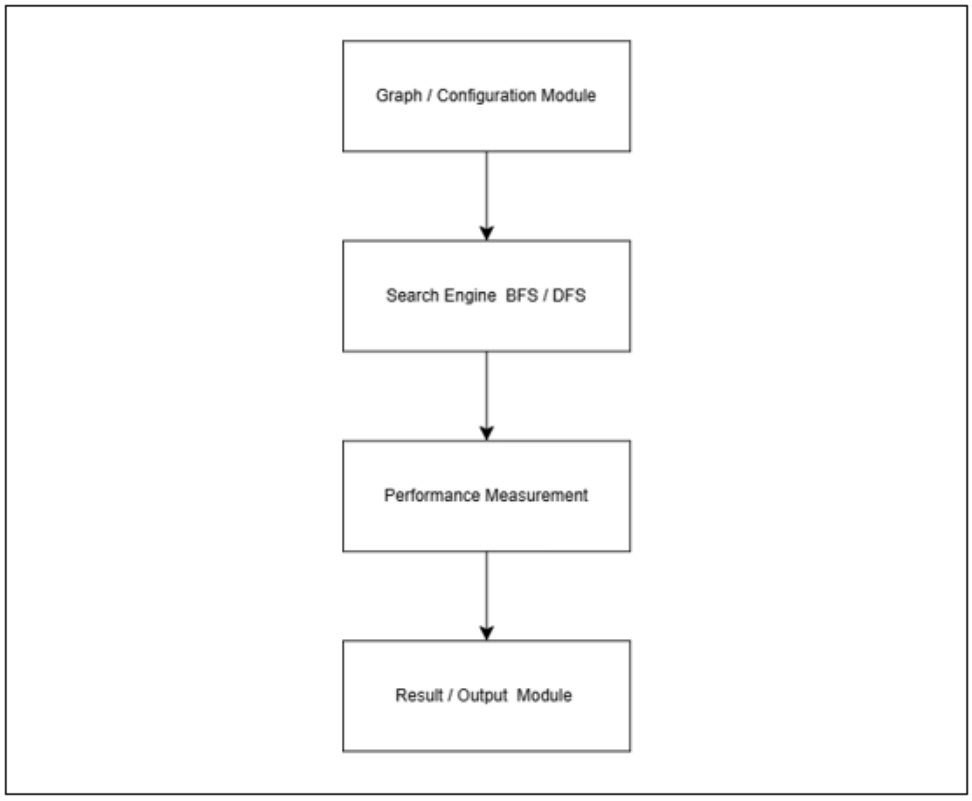


Figure 1: C4 Level 2 Container Diagram of the BFS vs DFS Search System

4. Rationale / Justification

I selected the modular design because both algorithms can use the same graph and input settings. In SLE-2, the average execution time was 9.45187 ms for BFS and 4.35503 ms for DFS. BFS expanded 20 nodes on average, while DFS expanded 6. Both algorithms found goal node 19. DFS performed better in this particular experiment, although results may differ for other graphs or goal positions. The SLE-3 C4 design separates graph/configuration, search, performance measurement and output. This structure helps explain, test and maintain each part without assuming DFS is always faster.

Metric	**BFS**	**DFS**
Average execution time	9.45187 ms	4.35503 ms
Average nodes expanded	20	6
Goal found	True	True

In my SLE-2 experiment, DFS had a lower average execution time and expanded fewer nodes than BFS. DFS took 4.35503 ms and expanded 6 nodes, while BFS took 9.45187 ms and expanded 20 nodes. Both algorithms found the goal. These results support comparing both algorithms using measured evidence. However, the results apply to this particular graph and do not prove that DFS is always faster. The modular architecture documented in SLE-3 helps separate and understand the different system responsibilities.

5. Consequences

Positive consequences:

- Both algorithms can be tested and compared using the same input.
- Separate modules make the system easier to understand and maintain.
- Execution time and expanded-node results provide practical evidence.

Negative consequences / Trade-offs:

- Maintaining two search algorithms requires additional code.
- Execution time can vary between runs.
- Results from this 20-node graph may not apply to every graph.

3. Part B — Short Summary of My SLE Journey

In SLE-1, I developed the search project. In SLE-2, I profiled BFS and DFS on a 20-node graph and recorded execution time and nodes expanded. In SLE-3, I described the system using the C4

Context, Container, Component and Code levels. In SLE-4, I documented the modular architecture decision and justified it using measured results and architecture design.

Part C — AI Contribution Note

- **AI tool used:** ChatGPT.
- **What AI helped with:** Understanding the ADR format and organizing explanations of BFS/DFS results and the C4 model.
- **What I did myself:** Ran the program, collected profiling results, generated the py-spy profile, checked the repository and prepared the architecture documentation.

6. References

1. SLE-2: Profiling Report – BFS vs DFS.
2. SLE-3: Architectural Design (Full C4 Model).
3. GitHub repository: https://github.com/pallavikolii285-ops/IAI_SLE2

7.Conclusion:

The SLE-2 and SLE-3 activities helped in understanding the performance and architecture of the BFS vs DFS Search System. In SLE-2, Breadth-First Search (BFS) and Depth-First Search (DFS) were tested on a graph containing 20 nodes, with node 0 as the starting point and node 19 as the goal. BFS took an average execution time of 9.45187 ms and expanded 20 nodes, while DFS took 4.35503 ms and expanded 6 nodes. Both algorithms successfully found the goal.

In SLE-3, the C4 model was used to represent the system at the Context, Container, Component, and Code levels. The architecture separated graph configuration, search algorithms, performance measurement, and result generation into different modules. Based on the observations, DFS performed faster in this particular experiment, although performance may vary depending on the graph structure and search requirements.

Overall, this work improved the understanding of algorithm performance, software architecture, modular design, and evidence-based architectural decision-making. The selected modular design makes the system easier to understand, test, maintain, and extend in the future.